

Algorithm for Bioinformatics

Exercise 7 David Riemer, Usama Mehmood

Problem 1

Due to the Handshaking Lemma in Graph Theory, this example is not possible. One of the conclusions of the lemma is, that every (regular) Graph like the one in the example has an even number of vertices with an uneven degree. So a graph $G = (V, E)$, with $V = 99$ and $d_G(v) = 17$ is not possible. In general $V * d_G(v)$ should be even.

Problem 2

From class we used the de Bruijn Algorithm to reconstruct the sequence of a Genome using an Eulerwalk.

When formulating the Question in Graph language, it comes out to if we can add edges to the de Bruijn Graph of R so the new Graph also has an eulerian walk. In order for it to be that, G has to be strongly connected, as well as have $d_{out}(v) - d_{in}(v) = 0 \forall v \in V$, or for all vertices, except 2, where $d_{out}(v) - d_{in}(v) = 1$ or -1 respectively.

So the idea our algorithm goes as follows

Step 1: Construct the de Bruijn graph from the multiset R of n k - mers:

1. Vertices are all $(k - 1)$ - mers appearing as prefixes or suffixes of the k - mers.
2. For each k -mer, add a directed edge from its prefix $(k - 1)$ - mer to its suffix $(k - 1)$ - mer.
3. Time: $O(nk)$

Step 2: Compute vertex imbalances:

1. Check Euler-Walk conditions on this graph
 - For every vertex compute $b(v) = d_{out}(v) - d_{in}(v)$
 - Sum up all vertices with $d_{out}(v) - d_{in}(v) > 0$ as P
2. Check if $P > t + 1$. If true, return not possible.

Time: $O(n + k)$

Step 3: Check weak connectivity:

Count the number C of weakly connected components that contain at least one edge.

If $C > t + 1$ return not impossible (cannot connect components with at most t added edges).

Time: $O(n + k)$.

Step 4: Identify relevant vertices:

Let V^* = all vertices with nonzero imbalance.

Size of V^* is at most $2(t + 1)$. These are the only vertices involved in fixing degrees.

5. Enumerate possible "balancing schemes"

Step 5: Enumerate balancing schemes:

1. For Eulerian trail, the final imbalances must be either all zero (Eulerian cycle) or exactly one vertex with $+1$ and one with -1 (Eulerian trail).
 2. For each choice of leaving at most one source (vertex with $b(v) < 0$) and one sink (vertex with $b(v) > 0$) with one unit of imbalance, compute the required number of edges F .
- (a) For each source v with $b(v) < 0$, let $\varepsilon_v \in \{0, 1\}$ be the unused supply, so the used supply is

$$a_v = |b(v)| - \varepsilon_v$$

(b) For each sink w with $b(w) > 0$, let $\delta_w \in \{0, 1\}$ be the unused demand, so the used demand is

$$b_w = b(w) - \delta_w$$

(c) Require

$$\sum_v \varepsilon_v = \sum_w \delta_w \in \{0, 1\}.$$

(d) Let

$$F = \sum_v a_v = \sum_w b_w.$$

If $F > t$, skip the scheme.

3. The number of such schemes is $O(|V^*|^2) = O(t^2)$.

Step 6: For each valid scheme, enumerate all multisets of F edges

Each added edge goes from a source ($b(v) < 0$) to a sink ($b(w) > 0$). Since the numbers of sources and sinks is $O(t)$, the number of such multisets is at most $O(t^t) = 2^{O(t \log t)}$.

Step 7: For each multiset E , check:

1. **Degree Condition:** For each source v , the number of edges in E starting at v equals a_v . For each sink w , the number of edges ending at w equals b_w .

2. **Connectivity Condition:** Construct an undirected graph H on the C components.

For each edge (v, w) in E , add an undirected edge joining the components containing v and w .

Check if H is connected.

3. **If both conditions hold, a solution exists.**

Construct the added $k - mers$: for each edge (v, w) , form a $k - mer$ by concatenating v with the last character of w .

The string A can be obtained by finding an Eulerian trail in the augmented graph.

If no multiset passes, return NO.

The total runtime of our algorithm is:

$$O(2^{O(t \log t)} + (n + k) + nk)$$